Fork, Explore, Commit: OS Primitives for Agentic Exploration

Cong Wang[1], Yusheng Zheng[2]

[1]Multikernel Technologies, Inc. · [2]UC Santa Cruz

Agentic OS Workshop, ASPLOS 2026

The Problem: Agents Have Side Effects

Agentic Exploration

- AI agents (SWE-Agent, OpenHands, Claude Code) execute **multi-step tasks**
- Shell commands, file edits, package installs → **irreversible side effects**
- Increasingly exploring **multiple paths in parallel**: Tree-of-Thoughts, Reflexion, Best-of-N

The Core Challenge

- Each path modifies **filesystem state** (workspace files)
- Each path spawns **processes** (compilers, test runners)
- Need to **isolate** each path, **commit** the winner, **discard** the rest

Example: A software engineering agent tries 3 candidate fixes simultaneously.
Only the one that passes tests should be committed — the rest must be cleanly discarded.

What Do Agents Use Today?

Approach	Limitation
Git stashing	Cannot capture non-tracked files (build artifacts, node_modules)
Temporary directories	Manual setup/teardown, no atomic commit
Container clones	Heavyweight, requires root, slow startup
Per-file snapshots	Miss shell command side effects, no parallel branching

None captures all filesystem modifications with atomic commit/rollback.

- **LangChain & AutoGPT**: rely on git stashing or temporary directories
- **Claude Code**: per-file snapshots that cannot capture `npm install` side effects

Six Requirements for Agentic Exploration

#	Requirement	Why
R1	**Isolated parallel execution**	Concurrent paths modify same files
R2	**Atomic commit + single-winner**	Apply winner's changes, invalidate siblings
R3	**Hierarchical nesting**	Tree-of-Thoughts explores sub-variants
R4	**Complete filesystem coverage**	Must capture *all* modifications, not just tracked files
R5	**Lightweight, unprivileged, portable**	Sub-ms creation, no root, any FS (ext4, XFS, NFS...)
R6	**Process coordination**	Reliable termination, sibling isolation

No existing OS mechanism satisfies all six requirements.

Gap: Existing Filesystem Mechanisms

Feature	OverlayFS	Btrfs/ZFS	DM-Snap	DAXFS	**BranchFS**
Portable across filesystems	✓	✗	✓	✗	✓
Nested branches	✗*	✓	✗	✓	✓
Commit-to-parent	✗	✗	✗	✓	✓
Sibling invalidation	✗	✗	✗	✓	✓
No root privileges	✗	✗	✗	✗	✓

- **OverlayFS**: stacked overlays since Linux 5.0, but no commit semantics, requires root
- **Btrfs/ZFS**: nested subvolumes, but filesystem-specific, no commit-to-parent
- **Device-mapper snapshots**: require raw block devices, O(depth) read latency
- **DAXFS**: branching with commit, but memory-backed only — not portable

Gap: Existing Process Management

Feature	pgrp	session	cgroup	PID ns	**branch()**
Reliable termination	✗	✗	✓	✓	✓
No escape possible	✗	✗	✓	✓	✓
Sibling isolation	✗	✗	✗	✓	✓
No setup required	✓	✓	✗	✗	✓
No root privileges	✓	✓	✗†	✓*	✓
No PID 1 complexity	✓	✓	✓	✗	✓

- **Process groups/sessions**: processes can escape via `setsid()` or `setpgid()`
- **Cgroups**: reliable termination, but require setup and typically root

Our Solution: The Branch Context

Definition

A **branch context** encapsulates:

1. A **filesystem view**: origin's files overlaid with a copy-on-write delta layer (Δ_i)
2. A **process group** whose side effects are confined to the context

Lifecycle

```
Fork ──▶ Explore ──▶ Commit (winner)
                 └──▶ Abort  (losers)
```

Created in **sets of N siblings** from a single frozen origin.

Four Core Semantic Properties

1. Frozen origin — parent becomes read-only; no merge conflicts by construction

2. Parallel isolated execution — N contexts run simultaneously, fully isolated

3. First-commit-wins — first to commit wins; siblings auto-invalidated

4. Nestable contexts — fork sub-contexts to form an exploration tree

Architecture Overview

```
        ┌──────────────────┐
        │ Parent Process   │
        └──────────────────┘
                │
                │ branch(N=3)
        ┌───────┼────────┐
        ▼       ▼        ▼
    ┌─────┐  ┌─────┐  ┌─────┐
    │ Ch 1 │ │ Ch 2 │ │ Ch 3 │   ← Process isolation
    └─────┘  └─────┘  └─────┘
       │        │        │
    ┌──▼──┐  ┌──▼──┐  ┌──▼──┐
    │Mt NS│  │Mt NS│  │Mt NS│     ← Mount namespace
    └─────┘  └─────┘  └─────┘
       │        │        │
    ═══╪═══════╪═══════╪═══
    │  BranchFS (FUSE)    │     ← Filesystem branching
    ═══╪═══════╪═══════╪═══
       │        │        │
    ┌──▼──┐  ┌──▼──┐  ┌──▼──┐
    │ Δ₁  │  │ Δ₂  │  │ Δ₃  │   ← Per-branch deltas (CoW)
    └─────┘  └─────┘  └─────┘
       └────────┼────────┘
                ▼
        ┌──────────────────┐
        │ Base Directory   │     ← Original files
        └──────────────────┘
```

BranchFS: FUSE-Based Copy-on-Write

File-Level CoW (~3,400 lines of Rust)

- First write → **copy entire file** to branch's delta layer
- Subsequent reads/writes served from **delta copy**
- Unmodified files resolved by walking the **branch chain**

Branch Chain Resolution

1. Check current branch's delta
2. Walk ancestor branches in order
3. Fall back to base directory
4. **Tombstone markers** prevent deleted files from reappearing

Key Design Properties

O(1) branch creation — just create a delta directory

No root privileges — runs entirely as a FUSE daemon

Portable — works on ext4, XFS, NFS, any FS

Commit & Abort

- **Commit**: copy delta to parent, increment **epoch counter** → invalidates all siblings
- **Abort**: discard delta layer → **near-zero cost**
- Cost proportional to **modification size**, not total FS size

BranchContext: Python Integration Library

7 Exploration Patterns

Pattern	Strategy
Speculate	Race N candidates, first success wins
BestOfN	Run N, commit highest-scoring
Reflexion	Sequential retry with feedback
TreeOfThoughts	Hierarchical nested branches
BeamSearch	Keep top-K at each depth
Tournament	Pairwise elimination via judge
Cascaded	Start with 1, fan out on failure

Usage Example

```python
ctx = BranchContext("/workspace")

# Best-of-N: try 3 fixes, commit best
results = ctx.best_of_n(
    n=3,
    task=lambda b: try_fix(b),
    score=lambda b: run_tests(b)
)
# Winner automatically committed
```

- Each pattern manages **branch lifecycle** + **process isolation** internally
- Agent code only supplies **per-branch task logic**

github.com/multikernel/branching

The branch() Syscall

Why a Kernel Primitive?

Setting up cgroups + PID ns + mount ns + FS branches in **userspace**:

- **Multi-step** with race windows between steps
- A process can fork between cgroup creation and migration
- **Error-prone** cleanup on partial failure

`branch()` composes all atomically in a **single call** with kernel-side cleanup on failure.

Three Operations

- `BR_CREATE` — fork N children, each in its own branch
- `BR_COMMIT` — apply FS changes, terminate siblings
- `BR_ABORT` — discard changes, terminate self

Interface

```
long branch(int op,
            union branch_attr *attr,
            size_t size);
```

Composable Flags

Flag	Effect
`BR_FS`	Mount namespace + FS branch
`BR_MEMORY`	Page-table CoW
`BR_ISOLATE`	Signal/ptrace barriers
`BR_CLOSE_FDS`	Close inherited FDs

- FS-agnostic: generic `FS_IOC_BRANCH_*` ioctls
- Adding a new branching FS = implement 3 ioctls

Evaluation: Branch Creation — O(1)

Creation Latency vs. Base Size

Base Size (files)	Latency
100	292 µs
1,000	317 µs
10,000	310 µs

Creation latency remains **under 350 µs**, confirming **O(1) cost** — independent of base filesystem size.

Branch creation only allocates a new delta directory.

Commit & Abort vs. Modification Size

Mod. Size	Commit	Abort
1 KB	317 µs	315 µs
100 KB	514 µs	365 µs
1 MB	2.1 ms	890 µs

Commit cost is proportional to **modified data volume**, not total FS size. Under **1 ms** for small changes.

Abort is even faster — just removes the delta layer.

Hardware: AMD Ryzen 5 5500U, 8 GB DDR4, NVMe SSD. Median of 10 trials. BranchFS: ~3,400 lines of Rust, FUSE 3.

Evaluation: I/O Throughput

Sequential Read/Write (50 MB file, 64 KB blocks)

Mode	Read	Write
Native filesystem	8,800 MB/s	576 MB/s
BranchFS (regular FUSE)	1,655 MB/s	631 MB/s
BranchFS (passthrough)	**7,236 MB/s** (82% native)	719 MB/s

Passthrough mode (`FOPEN_PASSTHROUGH`): bypasses the FUSE daemon for unmodified files → 82% of native read throughput

Write exceeds native: BranchFS treats `fsync` as no-op for ephemeral branches — durability enforced at commit time, not per-write

Agent workloads are dominated by **LLM API latency** (100 ms – 10 s).

Related Work

Isolation Mechanisms

- **Containers** (Docker): namespaces + cgroups — general containment, not branching
- **gVisor**: kernel-level sandboxing
- **lwCs**: lightweight intra-process contexts
- **Dune**: HW virtualization for user-level sandboxing
- **Capsicum**: capability-based sandboxing

OS Speculation & Transactions

- **Speculator**: speculative execution for distributed FS
- **TxOS**: OS-level ACID transactions

Key Differentiators

vs. Speculator: parallel N-way exploration (not sequential); competitive first-commit-wins (not owner-decided); local agent workloads (not distributed systems)

vs. TxOS: nestable (not flat); long-lived (not short); userspace FUSE (not deep kernel mods)

vs. Containers/VMs: lightweight CoW branching with commit/abort semantics — not general containment

Discussion and Future Work

Current Limitations

- **External side effects** (network, IPC) not rolled back on abort
- **Single-winner** semantics only — no multi-branch merge
- File-level CoW limitations: symlinks, hardlinks, special files
- **BR_MEMORY** not yet implemented

Broader Applicability

- `n_branches=1` → **try-and-rollback**
  - Package management: try upgrade, abort if it breaks
  - System configuration tuning

Future Directions

Effect gating — buffer network/IPC until commit; agent gateways (e.g., Agentry) provide natural interposition points

Multi-branch merge — union of non-overlapping changes; conflict detection for overlapping files

Implementation roadmap — prototype `branch()` on Linux 6.19; BR_FS + BR_ISOLATE first

Agent integration — most agents checkpoint via files → BR_FS-only covers most current use cases

Key Takeaways

1. **Agentic exploration** is a first-class OS concern: agents need isolated, parallel workspaces with atomic commit/rollback
2. **Branch context** = new OS abstraction with fork/explore/commit lifecycle and first-commit-wins resolution
3. **BranchFS** = working FUSE implementation: O(1) creation, no root, portable across filesystems
4. **BranchContext** = Python library with 7 ready-to-use exploration patterns
5. **branch() syscall** = proposed kernel primitive for atomic process + FS coordination

BranchFS: **github.com/multikernel/branchfs**

BranchContext: **github.com/multikernel/branching**

Thank You — Questions?

Cong Wang — cwang@multikernel.io

Yusheng Zheng — yzhen165@ucsc.edu

Agentic OS Workshop, ASPLOS 2026

BranchFS: github.com/multikernel/branchfs BranchContext: github.com/multikernel/branching